

SPRAWOZDANIE Z TESTÓW

Tutej

Aplikacja webowa do zarządzania osiedlem sąsiedzkim

Pliki testowe	20
Przypadki testowe	84
Warstwy testów	Backend · Frontend · E2E
Frameworki	Vitest · Playwright
Rok	2026

Spis treści

Spis treści.....	2
1. Wprowadzenie.....	3
2. Środowisko testowe.....	3
2.1 Technologie i frameworki.....	3
2.2 Strategia izolacji.....	3
3. Testy backendowe.....	5
3.1 auth.test.ts — uwierzytelnianie i autoryzacja.....	5
POST /api/auth/register.....	5
POST /api/auth/login.....	5
PUT /api/auth/me/password.....	5
3.2 events.test.ts — zarządzanie wydarzeniami.....	5
3.3 forums.test.ts — forum dyskusyjne.....	6
3.4 listings.test.ts — giełda sąsiedzka.....	6
3.5 neighborhoods.test.ts — osiedla.....	6
3.6 notices.test.ts — ogłoszenia.....	6
4. Testy frontendowe komponentów.....	8
4.1 EventCard.test.tsx — karta wydarzenia.....	8
Scroll lock (efekt uboczny useEffect).....	8
Autoryzacja widoczności przycisku usuwania.....	8
Formatowanie ceny.....	8
4.2 ListingCard.test.tsx — karta ogłoszenia.....	8
Asynchroniczne mutacje stanu z Deferred Promise.....	8
Reaktywność na zewnętrzne zmiany danych.....	9
"Brudne dane" — cena jako string.....	9
4.3 NoticeCard.test.tsx — karta ogłoszenia osiedlowego.....	9
Algorytm dat relatywnych.....	9
Inicjały vs zdjęcie autora.....	9
Cleanup useEffect przy odmontowaniu.....	9
4.4 useAuth.test.ts — hooki autoryzacji.....	9
Parsowanie i odporność JWT.....	9
Nagłówki sieciowe i wylogowanie.....	9
4.5 InputField.test.tsx — pole tekstowe.....	9
4.6 SearchableSelect.test.tsx — wybór osiedla.....	10
5. Testy end-to-end (Playwright).....	11
5.1 events.spec.ts — podstrona wydarzeń.....	11
5.2 forum.spec.ts — lista wątków.....	12
5.3 home.spec.ts — dashboard.....	12
5.4 login.spec.ts — logowanie.....	12
5.5 notices.spec.ts — ogłoszenia.....	12
5.6 register.spec.ts — rejestracja.....	12
5.7 settings.spec.ts — ustawienia konta.....	12
5.8 trade.spec.ts — giełda sąsiedzka.....	12
6. Podsumowanie.....	13
6.1 Zastosowane techniki testowania.....	13

1. Wprowadzenie

Niniejsze sprawozdanie dokumentuje pokrycie testowe aplikacji Tutej — systemu webowego służącego mieszkańcom osiedla do zarządzania ogłoszeniami, wydarzeniami, forum dyskusyjnym oraz giełdą sąsiedzką.

Testy zostały zorganizowane w trzech warstwach odpowiadających architekturze aplikacji:

- Warstwa backendowa — testy jednostkowe i integracyjne routerów Express.js przy użyciu Vitest oraz Supertest.
- Warstwa frontendowa — testy komponentów React oraz hooków przy użyciu Vitest, React Testing Library i biblioteki userEvent.
- Warstwa end-to-end — testy przepływów użytkownika w prawdziwej przeglądarce przy użyciu Playwright z mockowaniem sieci.

W każdej warstwie przyjęto podejście izolacji od zewnętrznych zależności: baza danych Prisma jest zastępowana mockami in-memory, sieć blokowana przez Playwright route interception, a autoryzacja JWT podmieniana przez middleware mock.

2. Środowisko testowe

2.1 Technologie i frameworki

Narzędzie	Wersja / wariant	Zastosowanie
Vitest	Najnowsza	Runner testów jednostkowych i integracyjnych
Supertest	—	Symulacja żądań HTTP do Express.js bez uruchamiania serwera
@testing-library/react	—	Renderowanie i asercje komponentów React w jsdom
@testing-library/user-event	—	Symulacja interakcji użytkownika (klawiatura, klik)
Playwright	—	Testy end-to-end w prawdziwej przeglądarce (Chromium)
@prisma/client (mock)	vi.mock	Izolacja warstwy bazy danych w testach backendowych
jose (mock)	vi.mock	Izolacja JWT — fałszywy token "sztuczny_token_jwt_123"
bcrypt (spy)	vi.spyOn	Kontrola wyniku porównania hashów haseł

2.2 Strategia izolacji

We wszystkich warstwach testów zastosowano następujące techniki izolacji od zewnętrznych zależności:

- Baza danych (Prisma): każdy plik testowy backendowy zawiera kompletny mock klasy PrismaClient z implementacją metod findMany, findUnique, create i update. Mocki implementują rzeczywistą logikę (sortowanie, filtrowanie) zamiast zwracać dane statyczne — co umożliwia testowanie logiki przekazywania parametrów przez router.
- Autoryzacja (JWT): middleware authenticate jest podmieniane przez vi.mock na funkcję wstrzykującą obiekt req.user bezpośrednio do żądania, omijając weryfikację tokenu.
- Sieć w testach e2e: Playwright route interception przechwytyje żądania HTTP i zwraca

skonfigurowane odpowiedzi JSON, eliminując zależność od działającego serwera.

- Czas systemowy: testy komponentu NoticeCard używają `vi.useFakeTimers()` i `vi.setSystemTime()` dla deterministycznego testowania logiki dat relatywnych.
- `localStorage`: wszystkie testy frontendowe czyszczą `localStorage` w `beforeEach`, a testy e2e używają `context.addInitScript()` do ustawiania stanu sesji przed inicjalizacją React.

3. Testy backendowe

Testy backendowe weryfikują logikę routerów Express.js w izolacji od bazy danych. Każdy test wykonuje rzeczywiste żądanie HTTP przez Supertest do aplikacji Express, a odpowiedź jest porównywana z oczekiwanym statusem i strukturą JSON.

Plik testowy	Testy	Narzędzia	Testowane funkcjonalności	Zastosowane mocki
auth.test.ts	3	Vitest + Supertest	Rejestracja, logowanie, zmiana hasła	bcrypt.compare mock, jose JWT mock, Prisma user mock
events.test.ts	4	Vitest + Supertest	Pobieranie zdarzeń (sortowanie), tworzenie zdarzenia	Prisma event + user mock, rzutowanie typów
forums.test.ts	4	Vitest + Supertest	Pobieranie postów (search), usuwanie postów (autoryzacja)	Prisma forum/post mock, auth middleware mock
listings.test.ts	5	Vitest + Supertest	Tworzenie ogłoszenia, walidacja statusu	Prisma listing + user mock, enum walidacja
neighborhoods.test.ts	1	Vitest + Supertest	Pobieranie osiedli (sortowanie alfabetyczne)	Prisma neighborhood mock z warunkowym sortowaniem
notices.test.ts	5	Vitest + Supertest	Pobieranie ogłoszeń (filtr), szczegóły, tworzenie	Prisma announcement mock, auth middleware mock

3.1 auth.test.ts — uwierzytelnianie i autoryzacja

Plik testuje trzy kluczowe ścieżki krytyczne systemu autoryzacji. W każdym teście biblioteka bcrypt jest szpiegowana metodą vi.spyOn z przywracaniem stanu po teście (mockRestore), co gwarantuje izolację między przypadkami.

POST /api/auth/register

Testowany scenariusz: próba rejestracji na zajęty adres e-mail. Mock Prisma zwraca istniejący rekord dla adresu zajety@test.pl, co powinno wywołać odpowiedź 409 z komunikatem "Email już istnieje.". Test weryfikuje działanie bloku warunkowego if (existing) w routerze.

POST /api/auth/login

Testowany scenariusz: podanie błędnego hasła dla istniejącego konta. bcrypt.compare jest szpiegowany i wymuszony do zwrócenia false. Test sprawdza kod 401 oraz brak pola token w odpowiedzi — token nie może się wygenerować przy negatywnej weryfikacji kryptograficznej.

PUT /api/auth/me/password

Testowany scenariusz: podanie błędnego obecnego hasła przy zmianie. Endpoint wymaga tokenu Bearer (symulowanego przez mock jose), a bcrypt.compare jest wymuszony do false. Test oczekuje kodu 400 z komunikatem "Nieprawidłowe obecne hasło."

3.2 events.test.ts — zarządzanie wydarzeniami

Plik zawiera cztery testy weryfikujące logikę routera wydarzeń. Szczególnie istotny jest test

sortowania — mock Prismy celowo zwraca dane w odwróconej kolejności gdy nie wykryje parametru `orderBy: { date: 'asc' }`, co wymusza przekazanie przez router poprawnego zapytania do bazy.

GET /api/events — sortowanie chronologiczne:

Test sprawdza, czy pierwsza pozycja w odpowiedzi ma wcześniejszą datę niż druga. Bez przekazania `orderBy` mock zwróciłby dane w złej kolejności i test by nie przeszedł.

POST /api/events — rzutowanie typów i źródło `neighborhoodId`:

Test wysyła `authorId` i `price` jako stringi. Asercje sprawdzają, czy w odpowiedzi są liczbami (`parseInt` / `parseFloat`). Kluczowa asercja: `neighborhoodId` w odpowiedzi to 77 — wartość pobrana z bazy na podstawie `authorId`, nie z `body requestu`.

POST /api/events — brakujące pole i nieistniejący autor:

Dwa testy brzegowe — brak pola `image` zwraca 400, nieistniejący `authorId` (9999) zwraca 404.

3.3 forums.test.ts — forum dyskusyjne

Plik testuje logikę wyszukiwania postów oraz kontrolę dostępu przy usuwaniu. Kluczowa technika: zmienna `mockUser` jest modyfikowana między testami, co pozwala symulować różne role użytkownika bez ponownego tworzenia mocków.

GET /api/forums/:forumId/posts?search=klucze:

Mock Prismy implementuje filtrowanie — jeśli router przekaże `where.OR`, wyniki są filtrowane po tytule i treści. Test oczekuje zwrócenia tylko jednego posta pasującego do frazy.

DELETE /api/forums/posts/:postId — trzy scenariusze autoryzacji:

- Właściciel posta (`authorId === req.user.id`) → 200
- Cudzy post, rola USER → 403 z komunikatem "Brak uprawnień"
- Cudzy post, rola ADMIN → 200 (bypass warunku)

3.4 listings.test.ts — giełda sąsiedzka

Analogicznie do `events` — testuje rzutowanie typów, przypisanie `neighborhoodId` z bazy, walidację pustej tablicy zdjęć oraz walidację enum statusu (`AVAILABLE` / `RESERVED` / `SOLD`). Test niedozwolonego statusu `"KUPIONE_PRZEZ_SASIADA"` weryfikuje instrukcję `["AVAILABLE", "RESERVED", "SOLD"].includes(status)`.

3.5 neighborhoods.test.ts — osiedla

Pojedynczy test weryfikuje sortowanie alfabetyczne listy osiedli. Mock zwraca dane w kolejności [Wilda, Jeżyce, Grunwald] gdy brak `orderBy`, a po przekazaniu `orderBy: { name: 'asc' }` sortuje do [Grunwald, Jeżyce, Wilda]. Test sprawdza pozycje bezpośrednio.

3.6 notices.test.ts — ogłoszenia

Plik pokrywa pięć scenariuszy: pobieranie wszystkich ogłoszeń, filtrowanie po `neighborhoodId` (`query param`), pobieranie szczegółów po ID, odpowiedź 404 dla nieistniejącego ID oraz

tworzenie ogłoszenia. W teście POST kluczowa asercja sprawdza, że zwrócone `neighborhoodId` wynosi 5 — wartość pochodzi z obiektu `req.user` wstrzykiwanego przez mock middleware, nie z `body`.

4. Testy frontendowe komponentów

Testy frontendowe renderują komponenty React w środowisku jsdom i weryfikują ich zachowanie poprzez interakcje użytkownika oraz asercje na strukturze DOM. Stosowana jest filozofia "testuj zachowanie, nie implementację" — testy nie sprawdzają stanu wewnętrznego Reacta, lecz widoczne efekty w interfejsie.

Plik testowy	Testy	Narzędzia	Testowane funkcjonalności
EventCard.test.tsx	5	Vitest + Testing Library + userEvent	Scroll lock (useEffect), autoryzacja przycisku delete, formatowanie ceny (darmowe / PLN)
ListingCard.test.tsx	4	Vitest + Testing Library + userEvent + fireEvent	Async fetch z Deferred Promise (blokowanie sieci), reaktywność na zewnętrzne zmiany (rerender), dirty data (price jako string)
NoticeCard.test.tsx	6	Vitest + Testing Library + vi.useFakeTimers	Algorytm relatywnych dat (formatDate), inicjały vs zdjęcie autora, cleanup useEffect przy unmount
useAuth.test.ts	7	Vitest + jsdom + localStorage	Parsowanie JWT (Base64), odporność na uszkodzony token, generowanie nagłówków Bearer, logout
InputField.test.tsx	4	Vitest + Testing Library + userEvent	Renderowanie ikon (letters, lock), przekazywanie wartości do onChange, odporność na brak callbacku
SearchableSelect.test.tsx	3	Vitest + Testing Library + fetch mock	Fetch danych z API, mapowanie id→value dla react-select, ekstrakcja ID w onChange

4.1 EventCard.test.tsx — karta wydarzenia

Scroll lock (efekt uboczny useEffect)

Test weryfikuje, czy otwarcie modala ustawia `document.body.style.overflow = "hidden"`, a zamknięcie przywraca "auto". Jest to test efektu ubocznego modyfikującego globalny stan DOM — nie stan wewnętrzny komponentu.

Autoryzacja widoczności przycisku usuwania

`localStorage.setItem("userId", "999")` symuluje innego użytkownika niż autor (id: 100). Test sprawdza, że przycisk delete nie istnieje w DOM. Drugi test odwraca scenariusz — zgadzające się ID powoduje wyrenderowanie przycisku i wywołanie `onDelete` z poprawnym `id=1`.

Formatowanie ceny

Dwa testy sprawdzają logikę potrójnego warunku: `price === 0` → "Darmowe", `price > 0` → "49 PLN". Wymagają otwarcia modala przez symulację kliknięcia, aby sekcja szczegółów stała się widoczna.

4.2 ListingCard.test.tsx — karta ogłoszenia

Asynchroniczne mutacje stanu z Deferred Promise

Test blokuje odpowiedź sieciową przez ręcznie kontrolowaną obietnicę (`resolveNetworkCall`). Po kliknięciu przycisku "ZAREZERWOWANE" — w trakcie oczekiwania na sieć — test sprawdza, że przycisk jest zablokowany (disabled). Po rozwiązaniu obietnicy sprawdza odblokowanie i wywołanie `onUpdate` z nowym statusem.

Reaktywność na zewnętrzne zmiany danych

Test używa `rerender()` z zaktualizowanym obiektem ogłoszenia (status: "SOLD"). Sprawdza, że `useEffect` poprawnie przepina stan wewnętrzny i zmienia badge z "Dostępne" na "Sprzedane".

"Brudne dane" — cena jako string

Komponent przyjmuje `price`: "2499.50" (string zamiast number). Test sprawdza poprawne wyświetlenie sformatowanej ceny przy użyciu wyrażenia regularnego `/2\s*499,5\s*PLN/`, które toleruje warianty formatowania `toLocaleString` w `jsdom`.

4.3 NoticeCard.test.tsx — karta ogłoszenia osiedlowego

Algorytm dat relatywnych

Trzy testy z `vi.setSystemTime()` ustawiają czas systemowy na wartości odpowiadające: mniej niż minutę po `createdAt` ("przed chwilą"), 25 minut po ("25 min temu") oraz kilka dni po (pełna data "1 czerwca 2026").

Inicjały vs zdjęcie autora

Test z `photo`: null sprawdza obecność tekstu "TN" (inicjały z `firstName`: "Tomasz", `lastName`: "Nowak"). Test z `photo` ustawionym na URL sprawdza obecność tagu `img` z poprawnym atrybutem `src` i brak inicjałów.

Cleanup `useEffect` przy odmontowaniu

Test klika kartę (`overflow` = "hidden"), następnie odmontowuje komponent przez `unmount()`. Sprawdza, że funkcja czyszcząca `useEffect` przywróciła `overflow` = "auto".

4.4 useAuth.test.ts — hooki autoryzacji

Parsowanie i odporność JWT

Test poprawnego tokenu: konstruuje fałszywy JWT (`dummyHeader.base64Payload.dummySignature`) z zakodowanym `payloadem`. `getCurrentUser()` powinien sparsować `payload` i zwrócić obiekt z `email` i `role`.

Test uszkodzonego Base64: token "header.To Nie Jest Poprawne Base64!!!.signature" powinien wywołać wyjątek w `atob()` — test sprawdza że funkcja łapie go (`try/catch`) i zwraca null bez crashowania aplikacji.

Test bez kropek: token "invalid-token-without-dots" — `getCurrentUser()` zwraca null po próbie dostępu do nieistniejącego indeksu.

Nagłówki sieciowe i wylogowanie

`getAuthHeaders()` z tokenem: sprawdza strukturę { `Authorization`: "Bearer xyz123", `Content-Type`: "application/json" }. Bez tokenu: sprawdza brak klucza `Authorization` w obiekcie — zapobiega wysłaniu nagłówka "Authorization: Bearer null".

`logout()`: sprawdza usunięcie zarówno "token", jak i "isAuth" z `localStorage`, oraz że `getToken()` zwraca null.

4.5 InputField.test.tsx — pole tekstowe

Cztery testy weryfikują: renderowanie domyślnej ikony (obecność `svg` w strukturze), obecność ścieżki SVG ikony lock (atrybut `d` zawiera "M8 11V7"), przekazywanie czystego stringa do `onChange` (nie obiektu `Event`), oraz odporność na brak callbacku `onChange` (opcjonalne wywołanie ?.).

4.6 SearchableSelect.test.tsx — wybór osiedla

Trzy testy: pobieranie danych z endpointu `/api/neighborhoods` i renderowanie opcji, obsługa błędu sieci (zniknięcie spinnera loading), oraz ekstrakcja czystego ID liczbowego (44) z wybranej opcji react-select do callbacku `onChange`.

5. Testy end-to-end (Playwright)

Testy e2e uruchamiają aplikację frontendową React w przeglądarce Chromium i wykonują pełne przepływy użytkownika. Wszystkie żądania sieciowe do API są przechwytywane przez Playwright route interception — aplikacja nigdy nie komunikuje się z prawdziwym serwerem ani bazą danych.

Stan sesji użytkownika jest ustawiany przez `context.addInitScript()` — kod JavaScript wykonywany w przeglądarce przed inicjalizacją React, co gwarantuje, że komponenty odczytują poprawny stan `localStorage` podczas montowania.

Plik testowy	Testy	Testowane scenariusze
events.spec.ts	5	Filtrowanie "Moje wydarzenia", walidacja formularza (brak zdjęcia), dodawanie wydarzenia, obsługa błędu serwera (500), anulowanie usunięcia (dialog)
forum.spec.ts	3	Stan ładowania (skeleton), fallbacki null (icon, _count), nawigacja po kliknięciu karty, obsługa błędu sieci (abort)
home.spec.ts	5	Loader przy <code>Promise.all</code> (blokowanie jednego endpointu), wyświetlanie tylko <code>noticesData[0]</code> , fallback pustej listy (events, notices), obsługa błędu sieci
login.spec.ts	4	Poprawne logowanie + zapis <code>localStorage</code> , błąd API (401), błąd sieci (abort), walidacja frontendowa (puste pola)
notices.spec.ts	3	Ukrywanie przycisku dla roli USER, dodawanie ogłoszenia przez COUNCILLOR z weryfikacją nagłówka <code>Authorization</code> , obsługa błędu sieci
register.spec.ts	5	Walidacja długości nazwiska, walidacja hasła (duża litera + cyfra), rozbicie <code>fullName</code> → <code>firstName/lastName</code> , błąd 409 (zajęty email), błąd sieci
settings.spec.ts	4	Blokada przy niezgodnych hasłach, blokada przy za krótkim hasle, aktualizacja profilu (PUT), zmiana hasła (brak <code>confirmPassword</code> w payloadzie)
trade.spec.ts	4	Filtr "Moje oferty", reset do "Wszystkie oferty", publikacja ogłoszenia (kontrakt POST + UI), blokada bez zdjęć

5.1 events.spec.ts — podstrona wydarzeń

Plik zawiera pięć testów. Kluczowe wzorce:

- Filtrowanie "Moje wydarzenia": kliknięcie przycisku filtra i weryfikacja że karta drugiego użytkownika (`authorId: 999`) znika z DOM, a własna pozostaje.
- Walidacja bez zdjęcia: przechwytywanie `window.dialog` i sprawdzanie komunikatu alertu — request POST nie może wyjść bez zdjęcia.
- Pełny przepływ dodawania: wypełnienie formularza, obsługa `FileChooser` (`setFiles` z fake buforem), weryfikacja nowej karty na liście.
- Anulowanie usunięcia: dialog jest odrzucany (`dismiss`), karta musi pozostać widoczna — DELETE nie może się wywołać.

5.2 forum.spec.ts — lista wątków

Test stanu ładowania blokuje odpowiedź API przez nierozwiązaną obietnicę. Dopóki sieć "wisi", skeleton `div[class="loadingState"]` musi być widoczny, a karty forum nie mogą istnieć w DOM. Test fallbacków weryfikuje obsługę null w danych (`icon: null` → `"🔍 _count: null` → `"0"`). Test nawigacji: kliknięcie karty → `expect(page).toHaveURL("../forum/12")`.

5.3 home.spec.ts — dashboard

Pięć testów pokrywających: loader przy `Promise.all` (blokowanie jednego z dwóch endpointów), wyświetlanie tylko `noticesData[0]` z tablicy dwóch elementów, fallbacki "Brak zaplanowanych wydarzeń." i "Brak nowych ogłoszeń." przy pustych tablicach, oraz brak crashu przy błędzie sieci obu endpointów.

5.4 login.spec.ts — logowanie

Test poprawnego logowania: konstruuje fake JWT z zakodowanym payloadem i porównuje dokładną wartość tokenu zapisanego w `localStorage` (nie `toContain`). Test pustych pól: flaga `fetchWasCalled` sprawdza, że request HTTP nie wychodzi przed przejściem walidacji frontendowej.

5.5 notices.spec.ts — ogłoszenia

Test uprawnień: dla roli USER przycisk "Dodaj nowe ogłoszenie" nie renderuje się w DOM. Dla roli COUNCILLOR przycisk jest widoczny i kliknięcie otwiera formularz. Test publikacji weryfikuje nagłówek `Authorization` w żądaniu POST (`headers["authorization"] === "Bearer councillor-token"`) wewnątrz route handlera.

5.6 register.spec.ts — rejestracja

Pomocnicza funkcja `selectNeighborhood` używa `toPass()` (ponowne próby asercji) ze względu na niestabilność dropdown `react-select` w testach e2e. Test rozbicia imienia: `interceptedPayload` z route handlera jest sprawdzany pod kątem `firstName: "Jan"`, `lastName: "Kowalski"` oraz `neighborhoodId` jako liczba 10 (nie string).

5.7 settings.spec.ts — ustawienia konta

Cztery testy: blokada wysyłki przy niezgodnych hasłach (`apiCalled === false`), blokada przy za krótkim hasle, weryfikacja payloadu PUT profilu (`firstName` zmieniony, `lastName` zachowany), oraz weryfikacja payloadu zmiany hasła — pole `confirmPassword` nie może znaleźć się w żądaniu API.

5.8 trade.spec.ts — giełda sąsiedzka

Testy filtrowania "Moje oferty" / "Wszystkie oferty" dla `userId: 99`. Test publikacji ogłoszenia weryfikuje kontrakt POST: `price` jako liczba (nie string), `images` jako tablica z co najmniej jednym elementem `base64`. Route jest rejestrowany z pełnym URL (`http://localhost:5000/api/listings`) zamiast wzorca `**` ze względu na cross-origin między portem 5173 a 5000.

6. Podsumowanie

Warstwa	Pliki	Przypadki testowe	Narzędzia
Backend (unit/integration)	6	22	Vitest, Supertest
Frontend (komponenty + hooki)	6	29	Vitest, Testing Library
End-to-end (przeglądarka)	8	~33	Playwright
ŁĄCZNIE	20	~84	3 narzędzia testowe

6.1 Zastosowane techniki testowania

- Dynamiczne mocki z logiką warunkową: mocki Prismy implementują rzeczywiste sortowanie i filtrowanie, zwracając "złą" odpowiedź przy braku poprawnych parametrów — co faktycznie wymusza logikę routera.
- Deferred Promise: ręczne sterowanie czasem rozwiązania obietnicy sieciowej pozwala testować stany pośrednie (loading, disabled) w komponentach asynchronicznych.
- Falszywy zegar (vi.useFakeTimers): deterministyczne testowanie algorytmów operujących na bieżącym czasie, niezależnie od daty uruchomienia testów.
- Route interception (Playwright): pełna kontrola nad odpowiedziami API bez uruchamiania serwera — możliwość symulowania błędów sieci, opóźnień i różnych wariantów danych.
- Inspekcja payloadu HTTP: przechwytywanie zawartości requestów POST/PUT w testach e2e i weryfikacja kontraktu API (typy danych, obecność/brak pól).
- Szpiegowanie funkcji (vi.spyOn): podmiana zachowania bcrypt.compare bez ingerencji w inne funkcje modułu, z obowiązkowym mockRestore po każdym teście.